คู่มือสำหรับผู้เริ่มต้น

Thrust

ไลบรารีอัลกอริทึมแบบขนานสำหรับ CUDA C++

เกี่ยวกับเอกสารนี้

คู่มือนี้เรียบเรียงเป็นภาษาไทยด้วยถ้อยคำของผู้จัดทำ โดยใช้เอกสารทางการของ NVIDIA CCCL (CUDA Core Compute Libraries) เป็นแหล่งอ้างอิงหลัก โค้ดตัวอย่างทุกชิ้นผู้จัดทำเขียนขึ้นเองและ**คอมไพล์และรันจริงแล้ว**บนเครื่องต่อไปนี้

- GPU: NVIDIA GeForce RTX 5060 Ti (สถาปัตยกรรม Blackwell, compute capability 12.0)
- CUDA Toolkit 13.4 ซึ่งมาพร้อม CCCL / Thrust เวอร์ชัน 3.4.2
- Windows 11 + Microsoft Visual Studio 18 (MSVC)

ไฟล์โค้ดอยู่ในโฟลเดอร์ `examples/` และสั่งคอมไพล์ทั้งหมดได้ด้วย `build.bat`

Contents

1 Thrust คืออะไร **2**

2 การติดตั้งและคอมไพล์ **2**

2.1 ไม่ต้องติดตั้งเพิ่ม 3

2.2 คำสั่งคอมไพล์ 3

2.3 สคริปต์คอมไพล์ตัวอย่างทั้งหมด (Windows) 3

3 Container: `host_vector` และ `device_vector` **4**

3.1 ตัวอย่างที่ 1: SAXPY และผลรวม 4

4 อัลกอริทึมพื้นฐาน **6**

4.1 ตัวอย่างที่ 2: จัดการข้อมูลการซื้อขาย 6

5 Lambda และ Functor **7**

6 Fancy iterator: คำนวณโดยไม่ต้องสร้างอาร์เรย์ **8**

6.1 ตัวอย่างที่ 3: ผลรวมกำลังสอง และ VWAP 8

7 ตัวเลขสุ่มและการจำลอง Monte Carlo **9**

7.1 ตัวอย่างที่ 4: ราคาเดินสุ่ม 100,000 เส้นทาง 10

8 Execution policy และการซิงโครไนซ์ **11**

9 ใช้ร่วมกับ CUDA แบบดั้งเดิม และการวัดเวลา **11**

9.1 ตัวอย่างที่ 5: ใช้ร่วมกับ kernel และเทียบความเร็วการเรียงลำดับ 12

10 เคล็ดลับประสิทธิภาพและข้อผิดพลาดที่พบบ่อย **13**

11 Thrust, CUB หรือเขียน kernel เอง? **14**

12 แบบฝึกหัด **14**

แหล่งอ้างอิง **15**

1 Thrust คืออะไร

สาระจากแหล่งอ้างอิง (สรุปความ) — NVIDIA CCCL documentation และ GitHub NVIDIA/cccl

- Thrust เป็นไลบรารีอัลกอริทึมแบบขนานระดับสูงสำหรับ C++ ใช้งานคล้ายไลบรารีมาตรฐานของ C++ (STL) และสามารถรันได้ทั้งบน GPU และบน CPU หลายแกน ผ่าน backend ที่เลือกได้ ได้แก่ CUDA, OpenMP และ TBB
- ปัจจุบัน Thrust เป็นส่วนหนึ่งของ **CCCL (CUDA Core Compute Libraries)** ร่วมกับอีกสองไลบรารี คือ **CUB** (ชิ้นส่วนระดับล่างสำหรับเขียน kernel เช่น การ reduce ระดับ block หรือ warp) และ **libcudacxx** (ไลบรารีมาตรฐาน C++ ที่ใช้ได้ทั้งในโค้ด host และ device)
- CCCL ทุกส่วนเป็นแบบ **header-only** ไม่ต้องคอมไพล์หรือลิงก์ไลบรารีแยก และ**ติดตั้งมาพร้อม CUDA Toolkit** แล้ว `nvcc` จะใส่ path ของ header ให้อัตโนมัติ
- CCCL รุ่น 3.x รองรับ C++17 และ C++20 และใช้กับ CUDA Toolkit 12.x และ 13.x
- เวอร์ชันของ CCCL ต้องเท่ากับหรือใหม่กว่า CUDA Toolkit ที่ใช้เสมอ ห้ามใช้รุ่นที่เก่ากว่า

อธิบายแบบง่าย — เปรียบเทียบให้เห็นภาพ

ถ้าเคยเขียน C++ คงคุ้นกับ `std::vector`, `std::sort`, `std::accumulate` **Thrust คือสิ่งเดียวกัน แต่ทำงานบน GPU**

งาน	C++ STL (CPU)	Thrust (GPU)
อาร์เรย์ไดนามิก	`std::vector<T>`	`thrust::device_vector<T>`
เรียงลำดับ	`std::sort`	`thrust::sort`
หาผลรวม	`std::accumulate`	`thrust::reduce`
แปลงทีละตัว	`std::transform`	`thrust::transform`
ผลรวมสะสม	`std::inclusive_scan`	`thrust::inclusive_scan`

ข้อดีคือ**ไม่ต้องเขียน kernel เอง** ไม่ต้องคิดเรื่อง block/thread และไม่ต้อง `cudaMalloc`/`cudaMemcpy` เอง Thrust จัดการให้ และเลือกอัลกอริทึมที่เร็วให้ตามสถาปัตยกรรมของ GPU

สรุปสั้น ๆ — เมื่อไรควรใช้ Thrust

- งานที่เป็น “รูปแบบมาตรฐาน” กับข้อมูลจำนวนมาก: แปลงค่า, รวมค่า, เรียงลำดับ, กรอง, ผลรวมสะสม, จัดกลุ่มตาม key
- ต้องการเขียนเร็ว อ่านง่าย และได้ประสิทธิภาพดีโดยไม่ต้องจูน
- ถ้างานซับซ้อนมาก (เช่น ต้องใช้ shared memory เอง หรือแต่ละ thread มีลูปยาวที่มีตรรกะเฉพาะ) ให้เขียน kernel เอง แล้วใช้ Thrust เป็นตัวจัดการหน่วยความจำและงานรอบข้าง (ดูหัวข้อ 8)

2 การติดตั้งและคอมไพล์

2.1 ไม่ต้องติดตั้งเพิ่ม

ถ้าติดตั้ง CUDA Toolkit แล้ว ก็มี Thrust แล้ว ตรวจเวอร์ชันได้จาก macro `THRUST_VERSION` (เช่น `300402` = 3.4.2) ใน header `<thrust/version.h>` หรือดูไฟล์ใน `.../CUDA/v13.4/include/cccl/thrust/`

2.2 คำสั่งคอมไพล์

```
nvcc -std=c++17 -O3 --extended-lambda -arch=native my_program.cu -o my_program
```

ตัวเลือก	ความหมาย
`-std=c++17`	ใช้มาตรฐาน C++17 (ขั้นต่ำของ CCCL 3.x)
`-O3`	เปิดการปรับแต่งประสิทธิภาพของคอมไพเลอร์
`-extended-lambda`	อนุญาตให้ใช้ lambda ที่มี `__device__` หรือ `__host__ __device__` (จำเป็นถ้าส่ง lambda ให้ Thrust ทำงานบน GPU)
`-arch=native`	คอมไพล์ให้ตรงกับ GPU ในเครื่องที่คอมไพล์ (ถ้าจะนำไปรันเครื่องอื่น ให้ระบุ เช่น `-arch=sm_90`)

ข้อควรระวัง — บน Windows (MSVC) — พบจริงระหว่างทดสอบ

1. ต้องเปิดสภาพแวดล้อมของ Visual Studio ก่อน (เช่น “x64 Native Tools Command Prompt” หรือเรียก `vcvars64.bat`)
2. CCCL 3.x ต้องการ preprocessor แบบตามมาตรฐานของ MSVC ถ้าไม่ระบุจะคอมไพล์ไม่ผ่านพร้อมข้อความว่า “MSVC/cl.exe with traditional preprocessor is used” ให้เพิ่ม

   ```
   -Xcompiler /Zc:preprocessor
   ```

3. `vcvars64.bat` อาจเปลี่ยนโฟลเดอร์ปัจจุบัน ถ้าเขียนสคริปต์ `.bat` ให้เก็บ path ของโฟลเดอร์งานไว้ก่อนเรียก แล้ว `pushd` กลับมา (ดู `examples/build.bat`)

2.3 สคริปต์คอมไพล์ตัวอย่างทั้งหมด (Windows)

```
@echo off
REM Build and run all Thrust examples with nvcc on Windows.
REM Adjust VCVARS if your Visual Studio is installed elsewhere.
setlocal
set "HERE=%~dp0"
set "VCVARS=C:\Program Files\Microsoft Visual Studio\18\Community\VC\Auxiliary\Build\vcvars64.bat"
set "PATH=%PATH%;C:\Program Files (x86)\Microsoft Visual Studio\Installer"
call "%VCVARS%" >nul
REM vcvars64 may change the working directory, so go back explicitly
pushd "%HERE%"
set "FLAGS=-std=c++17 -O3 --extended-lambda -arch=native -Xcompiler /Zc:preprocessor"
for %%f in (01_basics 02_sort_scan 03_iterators 04_monte_carlo 05_interop_timing) do (
    echo ----- compiling %%f.cu
    nvcc %FLAGS% "%HERE%%%f.cu" -o "%HERE%%%f.exe" || exit /b 1
)
```

```
for %%f in (01_basics 02_sort_scan 03_iterators 04_monte_carlo 05_interop_timing) do (
    echo ===== %%f =====
    "%HERE%%%f.exe"
)
popd
endlocal
```

3 Container: host_vector และ device_vector

อธิบายแบบง่าย

- thrust::host_vector<T> เก็บข้อมูลใน**หน่วยความจำของ CPU** (เหมือน std::vector)
- thrust::device_vector<T> เก็บข้อมูลใน**หน่วยความจำของ GPU**
- การกำหนดค่าข้ามชนิด เช่น device_vector<float> d = h; คือ**การคัดลอกข้อมูลระหว่าง CPU กับ GPU** โดยอัตโนมัติ (เบื้องหลังคือ cudaMemcpy)
- เมื่อ vector หมดขอบเขต (scope) หน่วยความจำจะถูกคืนอัตโนมัติ ไม่ต้อง cudaFree

3.1 ตัวอย่างที่ 1: SAXPY และผลรวม

```cpp
// 01_basics.cu : host_vector / device_vector, copy, transform, reduce
#include <thrust/host_vector.h>
#include <thrust/device_vector.h>
#include <thrust/transform.h>
#include <thrust/reduce.h>
#include <thrust/sequence.h>
#include <cuda/std/functional>
#include <cstdio>

int main() {
    // 1) create data on the host (CPU)
    thrust::host_vector<float> h_x(5);
    for (int i = 0; i < 5; ++i) h_x[i] = static_cast<float>(i + 1); // 1 2 3 4 5

    // 2) copy to the device (GPU) with a simple assignment
    thrust::device_vector<float> d_x = h_x;
    thrust::device_vector<float> d_y(5, 10.0f); // 10 10 10 10 10
    thrust::device_vector<float> d_z(5);

    // 3) z = 2*x + y (SAXPY) using a device lambda
    const float a = 2.0f;
    thrust::transform(d_x.begin(), d_x.end(), d_y.begin(), d_z.begin(),
                      [=] __host__ __device__ (float x, float y) { return a * x + y; });

    // 4) sum of z on the GPU
    float total = thrust::reduce(d_z.begin(), d_z.end(), 0.0f, cuda::std::plus<float>());

    // 5) copy back and print
    thrust::host_vector<float> h_z = d_z;
```

```
    for (int i = 0; i < 5; ++i) std::printf("z[%d] = %.1f\n", i, h_z[i]);
    std::printf("sum(z) = %.1f\n", total); // expected 80.0
    return 0;
}
```

ผลลัพธ์ที่ได้จากการรันจริง

```
z[0] = 12.0
z[1] = 14.0
z[2] = 16.0
z[3] = 18.0
z[4] = 20.0
sum(z) = 80.0
```

อ่านโค้ดทีละส่วน

- `d_x = h_x`: คัดลอก 1, 2, 3, 4, 5 ไปยัง GPU
- `d_y(5, 10.0f)`: สร้างเวกเตอร์ 5 ตัวบน GPU ค่าเริ่มต้นเป็น 10 ทุกตัว
- `thrust::transform(x_begin, x_end, y_begin, out_begin, op)`: คำนวณ `out[i] = op(x[i], y[i])` สำหรับทุก i พร้อมกันบน GPU
- `[=] __host__ __device__ (...)`: lambda ที่ใช้ได้ทั้งบน CPU และ GPU (`[=]` คือคัดลอกตัวแปร `a` เข้าไปใน lambda)
- `thrust::reduce`: รวมค่าทั้งหมด ได้ $12 + 14 + 16 + 18 + 20 = 80$ และ**คืนค่ากลับมาที่ CPU** ทันที

ข้อควรระวัง — อย่าเข้าถึง device_vector ทีละตัวในลูปบน CPU

คำสั่งอย่าง `d_vec[i]` ใช้ได้ แต่**ทุกครั้งคือการคัดลอกข้อมูลข้าม PCIe หนึ่งครั้ง** ถ้าทำในลูปล้านรอบจะช้ามาก ให้คัดลอกทั้งก้อนกลับมาเป็น `host_vector` ครั้งเดียวแทน

4 อัลกอริทึมพื้นฐาน

อัลกอริทึม	ทำอะไร
`fill`, `sequence`	เติมค่าคงที่ / เติม 0, 1, 2, …
`copy`, `copy_if`	คัดลอก / คัดลอกเฉพาะตัวที่ผ่านเงื่อนไข (การกรอง)
`transform`	ใช้ฟังก์ชันกับทุกตัว (1 หรือ 2 อินพุต)
`reduce`	รวมทุกตัวเป็นค่าเดียว (ผลรวม, ค่ามาก, ค่าน้อย)
`transform_reduce`	แปลงแล้วรวมในขั้นเดียว (ไม่ต้องสร้างอาร์เรย์กลาง)
`inner_product`	$\sum x_i y_i$
`count`, `count_if`	นับตัวที่เท่ากับค่าหนึ่ง / ผ่านเงื่อนไข
`min_element`, `max_element`	หาตำแหน่งของค่าน้อยสุด / มากสุด
`sort`, `stable_sort`	เรียงลำดับ
`sort_by_key`	เรียง key และย้าย value ตามไปด้วย
`inclusive_scan`, `exclusive_scan`	ผลรวมสะสม (prefix sum)
`reduce_by_key`	รวมค่าแยกตามกลุ่มของ key ที่ติดกัน
`unique`, `remove_if`	ลบตัวซ้ำที่ติดกัน / ลบตัวที่ผ่านเงื่อนไข
`lower_bound`, `upper_bound`	ค้นหาแบบ binary search หลายค่าพร้อมกัน

4.1 ตัวอย่างที่ 2: จัดการข้อมูลการซื้อขาย

```cpp
// 02_sort_scan.cu : sort, sort_by_key, inclusive_scan, count_if
#include <thrust/device_vector.h>
#include <thrust/host_vector.h>
#include <thrust/sort.h>
#include <thrust/scan.h>
#include <thrust/count.h>
#include <cstdio>

int main() {
    // trade prices and their timestamps (deliberately out of order)
    float h_px[] = {100.5f, 99.8f, 100.1f, 101.0f, 99.9f};
    long h_ts[] = {30, 10, 20, 50, 40};
    float h_qty[] = {2.0f, 1.0f, 3.0f, 1.0f, 4.0f};

    thrust::device_vector<float> px(h_px, h_px + 5);
    thrust::device_vector<long> ts(h_ts, h_ts + 5);
    thrust::device_vector<float> qty(h_qty, h_qty + 5);

    // sort quantities by timestamp (keys = ts, values = qty)
    thrust::sort_by_key(ts.begin(), ts.end(), qty.begin());

    // cumulative traded volume over time
    thrust::device_vector<float> cumvol(5);
    thrust::inclusive_scan(qty.begin(), qty.end(), cumvol.begin());

    // how many trades were above 100.0 ?
    int n_above = thrust::count_if(px.begin(), px.end(),
                                   [] __host__ __device__ (float p) { return p > 100.0f; });
```

```
    // plain sort of prices
    thrust::sort(px.begin(), px.end());

    thrust::host_vector<long> t = ts;
    thrust::host_vector<float> v = cumvol, p = px;
    for (int i = 0; i < 5; ++i)
        std::printf("ts=%ld cum_volume=%.1f\n", t[i], v[i]);
    std::printf("sorted prices: %.1f %.1f %.1f %.1f %.1f\n", p[0], p[1], p[2], p[3], p[4]);
    std::printf("trades above 100: %d\n", n_above); // expected 3
    return 0;
}
```

ผลลัพธ์ที่ได้จากการรันจริง

```
ts=10 cum_volume=1.0
ts=20 cum_volume=4.0
ts=30 cum_volume=6.0
ts=40 cum_volume=10.0
ts=50 cum_volume=11.0
sorted prices: 99.8 99.9 100.1 100.5 101.0
trades above 100: 3
```

อ่านโค้ดทีละส่วน

- `sort_by_key(ts, qty)`: เรียงเวลาจากน้อยไปมาก และ**ย้ายปริมาณตามไปด้วย** ได้ปริมาณเรียงตามเวลา 1, 3, 2, 4, 1
- `inclusive_scan`: ผลรวมสะสม $1, 1+3, 1+3+2, \ldots = 1, 4, 6, 10, 11$ คือปริมาณซื้อขายสะสมตามเวลา
- `count_if`: นับราคาที่มากกว่า 100 ได้ 3 ตัว (100.5, 100.1, 101.0)

ข้อควรระวัง

ในตัวอย่างนี้ `px` ไม่ได้ถูกเรียงไปพร้อมกับ `ts` เพราะ `sort_by_key` ย้ายได้แค่ value ชุดเดียว ถ้าต้องการย้ายหลายอาร์เรย์ตาม key เดียวกัน ให้ใช้ `zip_iterator` เป็น value (หัวข้อ 6) หรือเรียงอาร์เรย์ดัชนีแล้วใช้ `gather`

5 Lambda และ Functor

ฟังก์ชันที่ส่งให้อัลกอริทึมของ Thrust (เช่น ใน `transform`) ต้อง**เรียกได้บน GPU** มีสองวิธี

วิธีที่ 1: functor (struct ที่มี `operator()`)

```
struct square {
    __host__ __device__ float operator()(float x) const { return x * x; }
};
thrust::transform(v.begin(), v.end(), out.begin(), square{});
```

วิธีที่ 2: extended lambda (ต้องคอมไพล์ด้วย `-extended-lambda`)

```
thrust::transform(v.begin(), v.end(), out.begin(),
                  [] __host__ __device__ (float x) { return x * x; });
```

ข้อควรระวัง — พบจริงระหว่างทดสอบ: lambda แบบ __device__ อย่างเดียว

lambda ที่ประกาศเป็น `__device__` อย่างเดียว ใช้กับ `thrust::transform` ได้ แต่เมื่อใช้กับ `thrust::transform_reduce` บน CCCL 3.4 แล้ว**คอมไพล์ไม่ผ่าน** เพราะไลบรารีต้องรู้ชนิดของค่าที่ lambda คืนออกมาตั้งแต่ฝั่ง host ซึ่ง lambda แบบ `__device__` อย่างเดียวบอกไม่ได้

วิธีแก้: ใช้ `__host__ __device__` หรือใช้ functor แทน ตัวอย่างทุกชิ้นในคู่มือนี้จึงใช้ `__host__ __device__`

ข้อควรระวัง — ตัวดำเนินการสำเร็จรูป

ใน CCCL 3.x `thrust::plus`, `thrust::maximum` ฯลฯ ถูกประกาศว่าเลิกใช้ (deprecated) ให้ใช้ `cuda::std::plus<T>` จาก `<cuda/std/functional>` แทน (หรือ `cuda::maximum` จาก `<cuda/functional>`)

6 Fancy iterator: คำนวณโดยไม่ต้องสร้างอาร์เรย์

อธิบายแบบง่าย

iterator คือ “ตัวชี้ตำแหน่ง” ที่อัลกอริทึมใช้อ่านข้อมูล Thrust มี iterator พิเศษที่**สร้างค่าขึ้นมาเองขณะอ่าน** โดยไม่ต้องเก็บไว้ในหน่วยความจำ ช่วยประหยัดหน่วยความจำและเวลาอ่าน/เขียน

iterator	ให้ค่าอะไร
`counting_iterator<T>(a)`	$a, a+1, a+2, \ldots$ (เหมือนดัชนี ไม่ใช้หน่วยความจำ)
`constant_iterator<T>(c)`	$c, c, c, \ldots$
`transform_iterator(it, f)`	$f(\text{it}[0]), f(\text{it}[1]), \ldots$ คำนวณตอนอ่าน
`zip_iterator(tuple(a, b))`	$(a[0], b[0]), (a[1], b[1]), \ldots$ อ่านหลายอาร์เรย์พร้อมกัน
`permutation_iterator(v, idx)`	$v[\text{idx}[0]], v[\text{idx}[1]], \ldots$ อ่านตามดัชนีที่กำหนด
`discard_iterator`	ทิ้งค่าที่เขียนลงไป (ใช้เมื่อไม่ต้องการผลลัพธ์บางส่วน)

6.1 ตัวอย่างที่ 3: ผลรวมกำลังสอง และ VWAP

```
// 03_iterators.cu : counting_iterator, transform_reduce, zip_iterator
#include <thrust/device_vector.h>
#include <thrust/iterator/counting_iterator.h>
#include <thrust/iterator/zip_iterator.h>
#include <thrust/transform_reduce.h>
#include <thrust/inner_product.h>
#include <thrust/tuple.h>
#include <cuda/std/functional>
#include <cstdio>

int main() {
    const int N = 1000000;

    // sum of i^2 for i = 0..N-1 without allocating any input array
    thrust::counting_iterator<long long> first(0), last(N);
    long long sumsq = thrust::transform_reduce(
        first, last,
        [] __host__ __device__ (long long i) { return i * i; },
```

```
        0LL, cuda::std::plus<long long>());
    std::printf("sum of squares = %lld\n", sumsq);

    // VWAP = sum(price*qty) / sum(qty)
    float h_px[] = {100.0f, 100.2f, 99.9f, 100.1f};
    float h_qty[] = {5.0f, 2.0f, 3.0f, 10.0f};
    thrust::device_vector<float> px(h_px, h_px + 4), qty(h_qty, h_qty + 4);
    float notional = thrust::inner_product(px.begin(), px.end(), qty.begin(), 0.0f);
    float volume = thrust::reduce(qty.begin(), qty.end());
    std::printf("VWAP = %.4f\n", notional / volume);

    // the same notional with a zip_iterator (walks two arrays together)
    auto zb = thrust::make_zip_iterator(thrust::make_tuple(px.begin(), qty.begin()));
    auto ze = thrust::make_zip_iterator(thrust::make_tuple(px.end(), qty.end()));
    float notional2 = thrust::transform_reduce(
        zb, ze,
        [] __host__ __device__ (thrust::tuple<float, float> t) {
            return thrust::get<0>(t) * thrust::get<1>(t);
        },
        0.0f, cuda::std::plus<float>());
    std::printf("notional (inner_product) = %.2f, (zip) = %.2f\n", notional, notional2);
    return 0;
}
```

ผลลัพธ์ที่ได้จากการรันจริง

```
sum of squares = 333332833333500000
VWAP = 100.0550
notional (inner_product) = 2001.10, (zip) = 2001.10
```

ตัวอย่าง — ตรวจผลด้วยมือ

- $\sum_{i=0}^{N-1} i^2 = \frac{(N-1)N(2N-1)}{6}$ เมื่อ $N = 10^6$ ได้ 333,332,833,333,500,000 ตรงกับผลจาก GPU (ใช้ `long long` เพราะค่าเกินขอบเขตของ `int`)
- VWAP (Volume-Weighted Average Price) $= \frac{\sum p_i q_i}{\sum q_i} = \frac{500+200.4+299.7+1001}{20} = \frac{2001.1}{20} = 100.055$
- `counting_iterator` ทำให้คำนวณกับดัชนีล้านตัวได้**โดยไม่ต้องจองหน่วยความจำเลย**

7 ตัวเลขสุ่มและการจำลอง Monte Carlo

อธิบายแบบง่าย

Thrust มีตัวสร้างเลขสุ่มใน `<thrust/random.h>` ที่ใช้ได้ทั้งใน host และ device เช่น `default_random_engine`, `uniform_real_distribution`, `normal_distribution`

เทคนิคสำคัญ: ให้แต่ละเส้นทางจำลองใช้ seed เดียวกัน แต่เรียก `discard(path_id * n)` เพื่อ “ข้าม” ไปยังช่วงตัวเลขสุ่มที่ไม่ซ้ำกับเส้นทางอื่น ผลจึงทำซ้ำได้และแต่ละเส้นทางเป็นอิสระต่อกัน

รูปแบบการแบ่งงาน: ใช้ `counting_iterator` เป็น “หมายเลขเส้นทาง” แล้วให้ `transform` เรียก functor หนึ่งครั้งต่อเส้นทาง ภายใน functor เป็นลูปเวลาแบบลำดับ (เหมือนแนวคิด “หนึ่ง thread = หนึ่งเส้นทาง”)

7.1 ตัวอย่างที่ 4: ราคาเดินสุ่ม 100,000 เส้นทาง

```cpp
// 04_monte_carlo.cu : Monte Carlo of a random-walk mid-price with thrust::random
// One element of the counting range = one simulated path.
#include <thrust/device_vector.h>
#include <thrust/iterator/counting_iterator.h>
#include <thrust/transform.h>
#include <thrust/transform_reduce.h>
#include <thrust/reduce.h>
#include <thrust/random.h>
#include <cuda/std/functional>
#include <cmath>
#include <cstdio>

struct simulate_path {
    float s0, sigma, dt;
    int n_steps;
    unsigned int seed;

    __host__ __device__
    float operator()(unsigned int path_id) const {
        thrust::default_random_engine rng(seed);
        rng.discard(static_cast<unsigned long long>(path_id) * n_steps); // independent stream per path
        thrust::random::normal_distribution<float> z(0.0f, 1.0f);
        float s = s0;
        const float step = sigma * sqrtf(dt);
        for (int i = 0; i < n_steps; ++i) // time loop stays sequential
            s += step * z(rng);
        return s; // final mid-price of this path
    }
};

int main() {
    const int n_paths = 100000;
    const int n_steps = 200; // T = 1, dt = 0.005
    const float s0 = 100.0f, sigma = 2.0f, dt = 0.005f;

    thrust::device_vector<float> s_T(n_paths);
    thrust::transform(thrust::counting_iterator<unsigned int>(0),
                      thrust::counting_iterator<unsigned int>(n_paths),
                      s_T.begin(),
                      simulate_path{s0, sigma, dt, n_steps, 20u});

    float mean = thrust::reduce(s_T.begin(), s_T.end()) / n_paths;
    float var = thrust::transform_reduce(
                    s_T.begin(), s_T.end(),
                    [=] __host__ __device__ (float s) { return (s - mean) * (s - mean); },
                    0.0f, cuda::std::plus<float>()) / (n_paths - 1);

    std::printf("paths = %d\n", n_paths);
    std::printf("mean S_T = %.3f (theory 100)\n", mean);
    std::printf("std S_T = %.3f (theory sigma*sqrt(T) = 2)\n", std::sqrt(var));
    return 0;
```

```
}
```

ผลลัพธ์ที่ได้จากการรันจริง

```
paths = 100000
mean S_T = 100.007 (theory 100)
std S_T = 2.000 (theory sigma*sqrt(T) = 2)
```

ตัวอย่าง — ตรวจกับทฤษฎี

ราคาเดินสุ่ม $\mathrm{d}S = \sigma\,\mathrm{d}W$ เริ่มที่ 100 ด้วย $\sigma = 2$, $T = 1$ ทฤษฎีบอกว่า S_T มีค่าเฉลี่ย 100 และส่วนเบี่ยงเบนมาตรฐาน $\sigma\sqrt{T} = 2$ ผลจาก GPU ได้ 100.007 และ 2.000 ตรงกับทฤษฎี (ความคลาดเคลื่อนของค่าเฉลี่ยที่คาดได้คือ $2/\sqrt{100000} \approx 0.006$)

ข้อควรระวัง

`discard` ของ `default_random_engine` (minstd) ทำงานเร็ว แต่ถ้าต้องการคุณภาพตัวเลขสุ่มสูงหรือสร้างจำนวนมหาศาล ควรพิจารณา cuRAND แทน

8 Execution policy และการซิงโครไนซ์

อธิบายแบบง่าย

อาร์กิวเมนต์แรก (ไม่บังคับ) ของอัลกอริทึม Thrust คือ **execution policy** บอกว่าจะรันที่ไหนและอย่างไร

policy	ความหมาย
(ไม่ระบุ)	Thrust เลือกจากชนิดของ iterator: `device_vector` → GPU, `host_vector` → CPU
`thrust::host`	บังคับรันบน CPU
`thrust::device`	บังคับรันบน GPU (ใช้กับ raw pointer ที่ชี้หน่วยความจำ GPU)
`thrust::cuda::par.on(stream)`	รันบน CUDA stream ที่กำหนด
`thrust::cuda::par_nosync`	ไม่รอให้ GPU ทำเสร็จก่อนคืนการควบคุม (ถ้าทำได้)

เรื่องการรอ (synchronization): อัลกอริทึมที่**คืนค่ากลับมาที่ CPU** เช่น `reduce`, `count_if`, `inner_product` จะรอให้ GPU คำนวณเสร็จก่อนเสมอ ส่วนอัลกอริทึมที่เขียนผลลงหน่วยความจำ GPU อย่างเดียว เมื่อใช้ `par_nosync` อาจคืนการควบคุมทันที ต้องเรียก `cudaStreamSynchronize` หรือ `cudaDeviceSynchronize` ก่อนใช้ผลบน CPU

9 ใช้ร่วมกับ CUDA แบบดั้งเดิม และการวัดเวลา

อธิบายแบบง่าย

- **จาก Thrust ไป kernel ที่เขียนเอง:** `thrust::raw_pointer_cast(v.data())` ให้ pointer ธรรมดา (`float*`) ส่งเข้า kernel ได้
- **จากหน่วยความจำที่ `cudaMalloc` เอง ไปใช้กับ Thrust:** ห่อ pointer ด้วย `thrust::device_ptr<T>` หรือระบุ `thrust::device` เป็น policy

- **วัดเวลา GPU:** ใช้ cudaEvent และ**อุ่นเครื่อง (warm-up)** หนึ่งครั้งก่อน เพราะการเรียกครั้งแรกมีต้นทุนเริ่มต้น

9.1 ตัวอย่างที่ 5: ใช้ร่วมกับ kernel และเทียบความเร็วการเรียงลำดับ

```
// 05_interop_timing.cu : raw pointers <-> Thrust, custom kernel, timing host vs device sort
#include <thrust/device_vector.h>
#include <thrust/host_vector.h>
#include <thrust/device_ptr.h>
#include <thrust/sort.h>
#include <thrust/fill.h>
#include <thrust/reduce.h>
#include <thrust/random.h>
#include <thrust/execution_policy.h>
#include <cuda_runtime.h>
#include <algorithm>
#include <chrono>
#include <cstdio>

__global__ void scale_kernel(float* x, int n, float a) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < n) x[i] *= a;
}

int main() {
    // (A) Thrust vector -> raw pointer for a hand-written kernel
    thrust::device_vector<float> v(1000, 1.0f);
    float* raw = thrust::raw_pointer_cast(v.data());
    scale_kernel<<<(1000 + 255) / 256, 256>>>(raw, 1000, 3.0f);
    cudaDeviceSynchronize();
    std::printf("(A) sum after kernel = %.1f (expected 3000)\n", thrust::reduce(v.begin(), v.end()));

    // (B) raw cudaMalloc memory -> Thrust algorithm via device_ptr
    float* d_buf = nullptr;
    cudaMalloc(&d_buf, 1000 * sizeof(float));
    cudaMemset(d_buf, 0, 1000 * sizeof(float));
    thrust::device_ptr<float> p(d_buf);
    thrust::fill(p, p + 1000, 2.0f);
    std::printf("(B) sum via device_ptr = %.1f (expected 2000)\n", thrust::reduce(thrust::device, p, p +
        1000));
    cudaFree(d_buf);

    // (C) timing: sort 20M floats on CPU (std::sort) and GPU (thrust::sort)
    const int N = 20000000;
    thrust::host_vector<float> h(N);
    thrust::default_random_engine rng(20);
    thrust::uniform_real_distribution<float> u(0.0f, 1.0f);
    for (int i = 0; i < N; ++i) h[i] = u(rng);

    thrust::host_vector<float> h_copy = h;
    auto t0 = std::chrono::high_resolution_clock::now();
```

```
    std::sort(h_copy.begin(), h_copy.end());
    auto t1 = std::chrono::high_resolution_clock::now();
    double cpu_ms = std::chrono::duration<double, std::milli>(t1 - t0).count();

    thrust::device_vector<float> d = h; // copy not included in timing
    thrust::device_vector<float> warm(1000, 1.0f);
    thrust::sort(warm.begin(), warm.end()); // warm-up
    cudaEvent_t e0, e1;
    cudaEventCreate(&e0); cudaEventCreate(&e1);
    cudaEventRecord(e0);
    thrust::sort(d.begin(), d.end());
    cudaEventRecord(e1);
    cudaEventSynchronize(e1);
    float gpu_ms = 0.0f;
    cudaEventElapsedTime(&gpu_ms, e0, e1);

    thrust::host_vector<float> back = d;
    bool same = std::equal(back.begin(), back.end(), h_copy.begin());
    std::printf("(C) sort %d floats: CPU std::sort %.1f ms, GPU thrust::sort %.2f ms, speedup %.1fx,
        results equal: %s\n",
              N, cpu_ms, gpu_ms, cpu_ms / gpu_ms, same ? "yes" : "no");
    return 0;
}
```

ผลลัพธ์ที่ได้จากการรันจริง (RTX 5060 Ti)

```
(A) sum after kernel = 3000.0 (expected 3000)
(B) sum via device_ptr = 2000.0 (expected 2000)
(C) sort 20000000 floats: CPU std::sort 1474.5 ms, GPU thrust::sort 4.13 ms, speedup 356.6x, results
    equal: yes
```

ข้อควรระวัง — อ่านตัวเลขความเร็วอย่างระมัดระวัง

ตัวเลข 357 เท่านี้เทียบกับ `std::sort` **แกนเดียว** และ**ไม่รวม**เวลาคัดลอกข้อมูลไป/กลับ GPU ซึ่งสำหรับข้อมูล 80 MB ใช้เวลาหลายมิลลิวินาที ถ้าเทียบกับการเรียงแบบขนานหลายแกนบน CPU ช่องว่างจะลดลง และถ้าข้อมูลต้องคัดลอกไปกลับทุกครั้ง ประโยชน์จะลดลงมาก **หลักการ:** ให้ข้อมูลอยู่บน GPU ให้นานที่สุด ทำหลายขั้นตอนต่อเนื่องบน GPU แล้วค่อยคัดลอกผลสุดท้ายกลับ

10 เคล็ดลับประสิทธิภาพและข้อผิดพลาดที่พบบ่อย

สรุปสั้น ๆ — เคล็ดลับ

1. **ลดการคัดลอกระหว่าง CPU–GPU** ให้น้อยที่สุด
2. **รวมขั้นตอน (fusion):** ใช้ `transform_reduce` หรือ fancy iterator แทนการ `transform` ลงอาร์เรย์ชั่วคราวแล้วค่อย `reduce`
3. **ใช้ `device_vector` ซ้ำ** แทนการสร้างใหม่ในลูป (การจองหน่วยความจำ GPU มีต้นทุน)
4. **ใช้ `float`** ถ้าความแม่นยำพอ GPU ทั่วไปคำนวณ float เร็วกว่า double มาก

5. **ข้อมูลน้อยอย่าใช้ GPU:** ข้อมูลไม่กี่พันตัว CPU อาจเร็วกว่า เพราะต้นทุนเริ่ม kernel และคัดลอกข้อมูล
6. **วัดเวลาให้ถูก:** อุ่นเครื่องก่อน ใช้ `cudaEvent` และระบุเสมอว่ารวม/ไม่รวมการคัดลอกข้อมูล

ข้อควรระวัง — ข้อผิดพลาดที่พบบ่อย

- ลืม `-extended-lambda` ⇒ คอมไพล์ lambda ที่มี `__device__` ไม่ผ่าน
- ส่ง `std::vector` หรือ pointer ของ CPU ให้อัลกอริทึมที่คาดว่าจะรันบน GPU
- เข้าถึง `device_vector` ทีละตัวในลูปบน CPU (ช้ามาก)
- ใช้ `int` รวมค่าจำนวนมากจนล้น (ดูตัวอย่างที่ 3 ที่ใช้ `long long`)
- ไม่ตรวจข้อผิดพลาด: Thrust โยน exception เช่น `thrust::system_error` หรือ `std::bad_alloc` (หน่วยความจำ GPU ไม่พอ) ควรครอบด้วย `try/catch`

11 Thrust, CUB หรือเขียน kernel เอง?

	Thrust	**CUB**	**kernel เอง**
ระดับ	สูง (เหมือน STL)	กลาง–ต่ำ	ต่ำสุด
ความง่าย	ง่ายที่สุด	ต้อง จัดการ หน่วย ความ จำ ชั่วคราวเอง	ต้องคิดทุกอย่างเอง
การควบคุม	น้อย	มาก (ระดับ device, block, warp)	ทั้งหมด
เหมาะกับ	sort, scan, reduce, กรอง, จัดกลุ่ม	ชิ้นส่วนใน kernel ที่เขียนเอง, จูนประสิทธิภาพ	ตรรกะ เฉพาะ ทาง เช่น จำลอง ตลาดทีละเส้นทาง

อธิบายแบบง่าย — แนวทางสำหรับโครงงานจำลองการซื้อขาย

งานจำลองตลาดแบบ “หนึ่ง thread = หนึ่งเส้นทาง” ที่มีลูปเวลายาว เขียนเป็น kernel เอง หรือเป็น functor ใน `thrust::transform` (เหมือนตัวอย่างที่ 4) ก็ได้ ส่วนงาน**หลังการจำลอง** เช่น หาค่าเฉลี่ย ส่วนเบี่ยงเบนมาตรฐาน เปอร์เซ็นไทล์ของกำไร (ต้อง `sort`) หรือจัดกลุ่มผลตามพารามิเตอร์ (`reduce_by_key`) ใช้ Thrust จะง่ายและเร็วที่สุด

12 แบบฝึกหัด

1. ใช้ `transform` คำนวณผลตอบแทนแบบลอการิทึม $r_i = \ln(p_{i+1}/p_i)$ จากอาร์เรย์ราคา (ใบ้: ส่ง `p.begin()` และ `p.begin()+1` เป็นสองอินพุต)
2. ใช้ `transform_reduce` หาความผันผวน (ส่วนเบี่ยงเบนมาตรฐาน) ของผลตอบแทนจากข้อ 1
3. ใช้ `sort` แล้วหาเปอร์เซ็นไทล์ที่ 5 ของกำไรจากตัวอย่างที่ 4 (Value-at-Risk อย่างง่าย)
4. ใช้ `reduce_by_key` รวมปริมาณการซื้อขายแยกตามระดับราคา (สร้างสมุดคำสั่งซื้อขายอย่างง่ายจากรายการคำสั่ง)
5. ดัดแปลงตัวอย่างที่ 4 ให้จำลองกลยุทธ์ Avellaneda–Stoikov (ใช้สูตร (29) และ (30)) แล้วเทียบผลกับตารางที่ 1 ของบทความ

แหล่งอ้างอิง

- NVIDIA, *CUDA Core Compute Libraries (CCCL) documentation* — https://nvidia.github.io/cccl/
- NVIDIA, *CCCL repository (Thrust, CUB, libcudacxx)* — https://github.com/NVIDIA/cccl
- NVIDIA, *CUDA Toolkit* — https://developer.nvidia.com/cuda-toolkit
- NVIDIA, *CUDA C++ Programming Guide* — https://docs.nvidia.com/cuda/cuda-c-programming-guide/